Keywords: multi-agent reasoning strategy optimization meta-learning agentic AI

Strategist: Designing Agentic Reasoning at Scale

A Meta-Agent Builds a Growing Library of Executable Strategy Blueprints, Winning 24 of 30 Benchmarks at 48% Lower Cost

By Taiming Lu, Qi Xu, Zhigang Hua, Zhuang Liu

arXiv:2611.03618 cs.AI / cs.CL NeurIPS 2026 Poster

Strategist is a meta-agent that automatically designs inference strategies for new tasks by representing each strategy as executable Python code, building a reusable library of blueprints that grows across tasks. Evaluated on 30 reasoning benchmarks, Strategist wins 24 and achieves 8.5 absolute percentage points higher average accuracy than the strongest fixed-strategy baseline at 48% lower inference cost. Strategies transfer across model backends the meta-agent never optimized on.

AT A GLANCE

Fixed pipeline templates (chain-of-thought, tree-of-thought, debate) do not generalize across domains and require manual design.

The topology of multi-agent reasoning is a critical variable for performance; automating its design could yield consistent improvements across tasks without human effort.

Strategies are Python programs (blueprints); a meta-agent retrieves related blueprints, proposes candidates, evaluates them on a dev set, revises them, and stores all valid candidates in a library.

8.5pp accuracy improvement and 48% cost reduction over the best fixed strategy, on 24 of 30 benchmarks.

30 benchmarks spanning math, coding, science, and commonsense; transfer experiments to Gemini-2.5 backend.

The Big Picture

Multi-agent systems for reasoning typically rely on templates: chain-of-thought prompts the model to reason step-by-step, tree-of-thought expands multiple reasoning branches, and debate pits agents against one another. These structures are designed by researchers for specific task types and do not automatically adapt to new domains. Meanwhile, automated prompt engineering optimizes over discrete tokens but leaves the structural topology of the reasoning process fixed.

The key insight of Strategist is that the topology itself—how agents are orchestrated, how outputs are routed and verified—is a programmable object that can be optimized. Representing strategies as Python programs makes them composable, executable, and amenable to code-level search by a language model.

Why Existing Approaches Fall Short

Fixed templates (chain-of-thought, majority voting) are designed for average-case performance across broad task categories. They cannot capture the structural insights that are most effective for a specific task type, such as using a symbolic calculator for arithmetic sub-steps or running parallel hypotheses for spatial reasoning.

Prompt optimization methods (AutoPrompt, DSPy, APE) treat the strategy as a fixed function and optimize over its textual inputs, but do not change the number of agents, the verification steps, or the routing logic. They are limited to the structural assumptions baked into the initial template.

Human-designed multi-agent systems require expert knowledge and significant engineering time, and the resulting designs are rarely reusable across domains.

Core Mechanism: Strategy as Code

Each strategy in Strategist is a Python program, or *blueprint*, that takes a task description and input as arguments and returns an answer by orchestrating one or more LLM calls. A blueprint may:

- Decompose the problem into typed sub-problems and dispatch each to a specialized agent.
- Route outputs conditionally based on model confidence or consistency checks.
- Call helper functions from other library blueprints as sub-routines.
- Aggregate results via voting, scoring, or symbolic verification.

This representation makes strategies fully executable and inspectable, enables compositional reuse, and gives the meta-agent a structured search space that is richer than prompt tokens alone.

The Design Loop

Given a new task T and a small development set $\mathcal{D}$:

1. **Retrieve:** Encode T with a frozen text encoder and retrieve top-k blueprints from the library by cosine similarity.
2. **Propose:** The meta-agent generates 3–5 candidate blueprints as Python code, guided by retrieved examples and the target task description.
3. **Evaluate:** Execute each candidate on $\mathcal{D}$; record accuracy a_i and total token cost c_i.
4. **Revise:** The meta-agent reviews evaluation results and produces a final revised blueprint for T.

5. **Store:** All candidates and the final blueprint are added to the library with metadata \((a_i, c_i, \text{task-type tags})\).

The loop runs for 3 rounds per task, with early stopping if the best candidate exceeds a target accuracy of \(\alpha^* = 0.9\).

The Strategy Library

The library is initialized with 20 hand-designed baselines. After processing all 30 benchmarks in order, the library contains \(\sim\)150 blueprints. The compositional reuse mechanism means that blueprints are not monolithic: a later task (e.g., GPQA) can invoke the "verify-arithmetic" sub-routine discovered for GSM8K and the "multi-perspective synthesis" routine from debate-style blueprints.

What the Guarantee Says

Strategist provides no PAC-learning or regret guarantee. The empirical claim is competitive: on the 30-benchmark suite, the designed strategies outperform all fixed baselines (including the best oracle fixed strategy per task). No theoretical bound on the number of meta-agent iterations needed to find a good strategy is given. The transfer result (Section 7) is empirical and suggests cross-model generality, but the mechanism is not formally analyzed.

Experimental Findings

- **Accuracy (30 benchmarks):** Strategist wins 24 of 30; average improvement is **8.5pp** over the strongest fixed baseline (which is the oracle best-fixed-strategy per task).
- **Cost:** Despite higher quality, Strategist uses **48% fewer tokens** on average, because it avoids unnecessary LLM calls and uses cheaper model tiers for sub-tasks.
- **Library growth:** Average accuracy on a held-out set of 5 benchmarks improves by 3.1pp after processing the first 25 tasks, confirming compositional library benefit.
- **Transfer to Gemini-2.5:** Strategies discovered with GPT-6 improve Gemini-2.5 chain-of-thought accuracy by **6.1pp** on average, confirming cross-model transferability.

Ablations and Interpretation

Without retrieval: Removing library retrieval (proposing strategies from scratch) reduces accuracy by 4.4pp, confirming that the library enables meaningful compositional reuse.

Strategies as text vs. code: Replacing Python blueprints with natural-language strategy descriptions reduces accuracy by 2.9pp, as natural-language strategies are less precise and cannot directly invoke verified sub-routines.

Without iterative revision: Single-round (no revision step) performance is 3.7pp below the full 3-round loop, with diminishing returns in rounds 2 and 3.

Meta-agent backbone: Replacing GPT-6 as the meta-agent with GPT-5 reduces strategy quality by 2.1pp on average, but the benefit of strategy design over no design is maintained across all backbones tested.

Reference

Taiming Lu, Qi Xu, Zhigang Hua, Zhuang Liu. **Strategist: Designing Agentic Reasoning at Scale.** arXiv:2611.03618. NeurIPS 2026. https://arxiv.org/abs/2611.03618